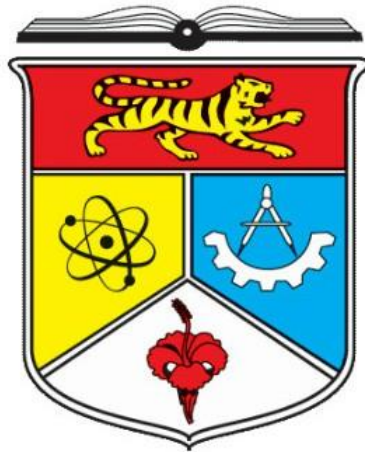


TTTC2343 Final Project Report



Autonomous Warehouse Inventory and Item Delivery Robot

Course	TTTC2343
Group Members	Danish Irfan bin Hakimi (A213696) Shukri Aizat bin Shuazrin (A212202) Ahmad Nazmi bin Bastian Affendy (A212353) Maheysh A/L Ramesh (A211881)
Project Title	Autonomous Warehouse Inventory and Item Delivery Robot Simulation

Abstract

This final report presents the complete development of an Autonomous Warehouse Inventory and Item Delivery Robot simulation for TTTC2343. The project was developed using ROS Noetic, Gazebo and RViz, with TurtleBot3 Burger used as the mobile robot platform. The simulation demonstrates a warehouse robot that starts from a charging or home station, navigates through a customized warehouse, visits shelf inventory zones, performs simulated inventory checking, travels to a packing or drop-off station, and completes a delivery mission.

The project follows the required workflow from proposal, Gazebo environment customization, SLAM mapping, autonomous navigation, UI-based destination control and task execution until final GitHub upload and video demonstration. The main outputs include a customized Gazebo world, ROS launch files, map files, navigation scripts, UI control script, mission controller, zone detector, task-action scripts, weekly documentation and final demo evidence. The final system satisfies the main project areas of technical methodology, system design, implementation, navigation and task execution results.

Table of Contents

1. Introduction
2. Project Objectives and Scope
3. Methodology
4. System Design
5. Implementation Details
6. Navigation and Task Execution
7. Results and Discussion
8. Issues, Limitations and Improvements
9. Conclusion
10. References
- Appendix A. Key Run Commands
- Appendix B. GitHub Submission Checklist

1. Introduction

Modern warehouses require fast item movement, accurate inventory checking and safe navigation around shelves, pallets and workers. Manual inventory checking and item delivery can become slow and repetitive, especially in a warehouse with many shelves and storage zones. A mobile robot can help reduce repetitive movement by travelling between storage areas, checking shelf zones and delivering items to a packing station.

This project develops a customized warehouse robot simulation using ROS and Gazebo. The simulation focuses on a warehouse assistant robot that can operate inside a structured environment with shelves, inventory zones, lane markings, a charging area and a drop-off station. The project was divided into weekly development stages: Week 7 proposal, Week 8 Gazebo environment customization, Week 9 mapping/navigation/UI development, Week 10 task action, and Week 11 final integration with GitHub upload and demo video.

The final system demonstrates the complete robotics pipeline required for the course: environment development, map generation, autonomous navigation, user interface control and simulated task execution. The system was implemented as a ROS package with launch files, Python nodes, a Gazebo world, saved map files and navigation/task scripts.

2. Project Objectives and Scope

The main aim of the project is to build and demonstrate a complete warehouse robot simulation that can support inventory checking and item delivery tasks. The project scope is limited to simulation in Gazebo and RViz, using ROS topics, launch files and Python nodes to control the robot and report task status.

2.1 Objectives

- To design a customized warehouse simulation environment with multiple areas, obstacles, shelves and task stations.
- To integrate a mobile robot platform into the Gazebo world and make it controllable using ROS topics.
- To perform mapping and generate a saved warehouse map for navigation.
- To implement multi-point navigation using warehouse checkpoints such as home, shelf zones, pickup zone and drop-off station.
- To create a simple UI that allows destination selection and shows robot operation status.
- To perform a simulated task action involving inventory verification and item delivery confirmation.
- To integrate all project files into a GitHub repository and provide final demo evidence.

2.2 Scope

The project is a simulation-based implementation. It does not use a physical robot. The task action is represented through marker-style inventory verification and delivery status messages rather than a real camera-based QR decoding pipeline or physical gripper pick-and-place. This is acceptable for the project demonstration because the focus is to show a working ROS/Gazebo robot system, navigation flow and task execution logic. Future work can add real QR code detection using OpenCV and camera topics.

3. Methodology

The methodology follows the course workflow: Gazebo environment development, SLAM mapping, autonomous navigation, task action and UI development, simulation demo, and GitHub submission. The same structure is used in this report to make the final implementation easy to verify against the marking rubric.

Final Project Workflow

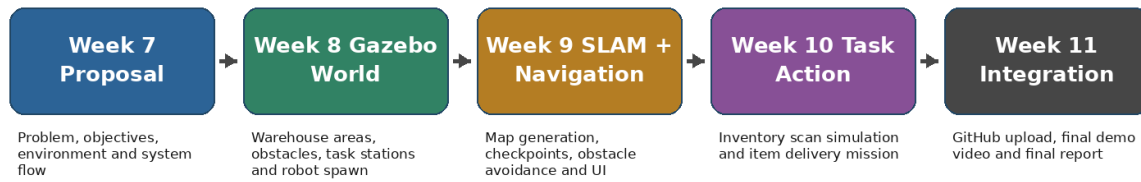


Figure 1. Overall development workflow from Week 7 to Week 11.

3.1 Development Phases

Week	Focus	Output
Week 7	Project Draft Proposal	Problem statement, objectives, environment idea, robot task and system flowchart.
Week 8	Gazebo Environment	Customized warehouse world with shelves, zones, obstacles, lane markings and robot integration.
Week 9	Mapping, Navigation and UI	SLAM map generation, saved map files, minimum three checkpoints, UI buttons and status display.
Week 10	Task Action	Simulated inventory verification and item delivery task in the warehouse environment.
Week 11	Final Integration	GitHub repository, full demo video and final report preparation.

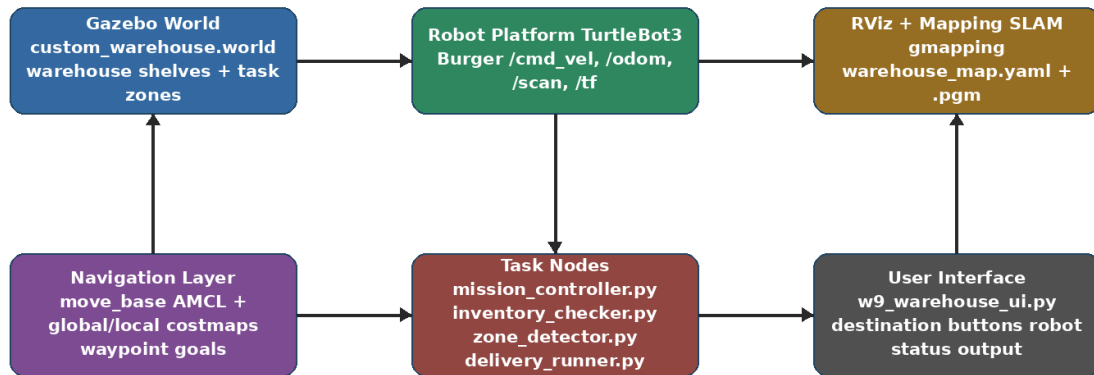
3.2 Software and Tools

Tool	Purpose
Ubuntu 20.04	Operating system used inside VirtualBox.
ROS Noetic	Middleware for packages, topics, nodes, launch files and robot control.
Gazebo	3D simulation environment for the warehouse and robot.
RViz	Visualization tool for SLAM map, robot pose, laser scan and navigation goals.
TurtleBot3 Burger	Mobile robot model used for simulation and autonomous movement.
Python 3 / rospy	Main programming language for mission controller, UI and task nodes.
GitHub	Repository platform for final submission and project integration.

4. System Design

The system is designed as a modular ROS project. The Gazebo world represents the warehouse environment, while the robot publishes sensor and motion information through ROS topics. The mapping and navigation modules use the robot scan, odometry and transform data. The UI and mission scripts send robot goals or velocity commands, while task nodes publish inventory and delivery status messages.

System Architecture



Main data and control flow: Gazebo provides the simulated warehouse and robot sensors. ROS topics carry odometry,

Figure 2. System architecture showing Gazebo, robot platform, mapping, navigation, task nodes and UI.

4.1 Repository and Package Structure

The GitHub repository stores the weekly reports, screenshots, videos and ROS source packages. The final Week 10 package is named `warehouse_robot`, while earlier development work used `warehouse_robot_danish`. The use of both names should be treated as development-to-final integration, not as separate project concepts.

File / Folder	Function
<code>warehouse_robot/worlds/custom_warehouse.world</code>	Final Gazebo warehouse world used for task demonstration.
<code>warehouse_robot/launch/warehouse_world.launch</code>	Loads the custom warehouse world in Gazebo.
<code>warehouse_robot/launch/spawn_robot.launch</code>	Spawns TurtleBot3 Burger into the world.
<code>warehouse_robot/launch/autonomous_mission.launch</code>	Starts world, robot, zone detector and mission controller.
<code>warehouse_robot/launch/warehouse_navigation.launch</code>	Starts navigation using the saved warehouse map.
<code>warehouse_robot/launch/navigation_waypoint_mission.launch</code>	Runs the optional move_base waypoint mission.
<code>warehouse_robot/scripts/mission_controller.py</code>	Controls the robot mission using <code>/cmd_vel</code> and logs scan/delivery progress.
<code>warehouse_robot/scripts/zone_detector.py</code>	Monitors odometry and publishes current warehouse zone status.
<code>warehouse_robot/scripts/inventory_checker.py</code>	Publishes simulated shelf inventory status.
<code>warehouse_robot/scripts/delivery_runner.py</code>	Publishes simulated delivery task status.

warehouse_robot/scripts/waypoint_mission.py	Sends multi-point navigation goals to move_base.
warehouse_robot/maps/warehouse_map.yaml and .pgm	Saved occupancy map used by navigation.

4.2 Main ROS Topics and Nodes

Node	Topic(s)	Role
warehouse_mission_controller	/cmd_vel	Publishes velocity commands and controls the demo mission sequence.
warehouse_zone_detector	/odom, /warehouse/current_zone	Reads robot odometry and reports which warehouse zone is nearest.
inventory_checker	/warehouse/inventory_status	Simulates shelf stock and marker verification messages.
delivery_runner	/warehouse/delivery_status	Simulates item pickup and delivery progress.
waypoint_mission	/move_base	Sends navigation goals to the ROS Navigation Stack.
Gazebo + TurtleBot3	/scan, /odom, /tf, /cmd_vel	Provides laser scan, odometry, transforms and movement interface.

5. Implementation Details

5.1 Customized Warehouse Environment

The Gazebo environment represents a warehouse with shelves, item boxes, lane markings, pickup and drop-off zones. The environment includes several functional areas, fulfilling the requirement for multiple areas or task zones. The robot starts near a home or charging zone and then moves toward shelf areas and delivery zones.

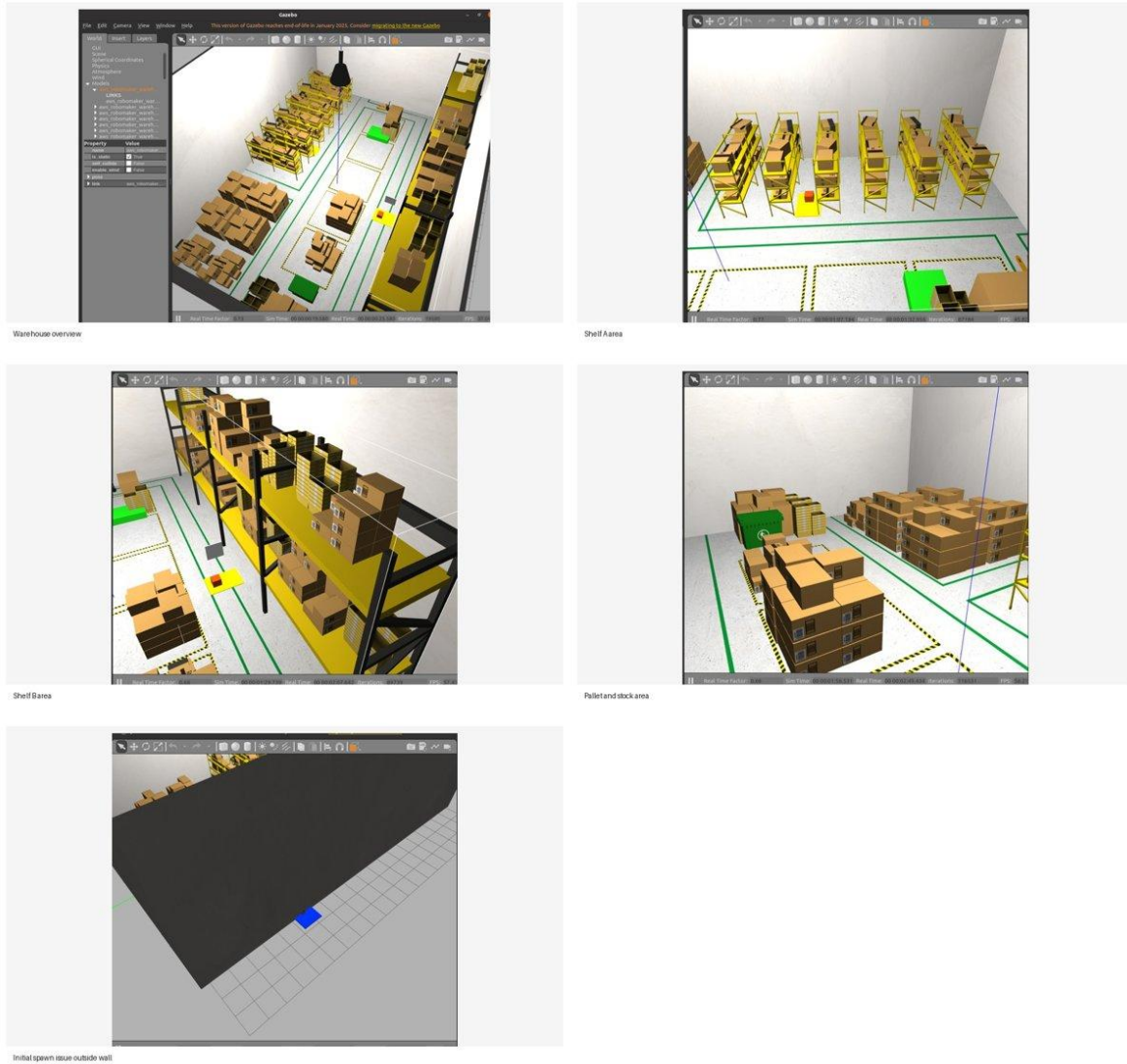


Figure 3. Customized warehouse environment, including shelves, storage areas, task zones and robot spawn area.

5.2 Robot Integration

The robot platform is TurtleBot3 Burger. It is spawned through the launch file `spawn_robot.launch`. The robot uses common ROS interfaces such as `/cmd_vel` for velocity control, `/odom` for odometry, `/scan` for laser range data and `/tf` for coordinate transforms. This makes the robot compatible with SLAM mapping, RViz visualization and `move_base` navigation.

5.3 Mapping Implementation

Mapping was performed using SLAM with the robot laser scan. During mapping, the robot was moved around the warehouse so that walls, shelves and major obstacles could be observed. The output was saved as `warehouse_map.pgm` and `warehouse_map.yaml`. These files are required for localization and navigation.

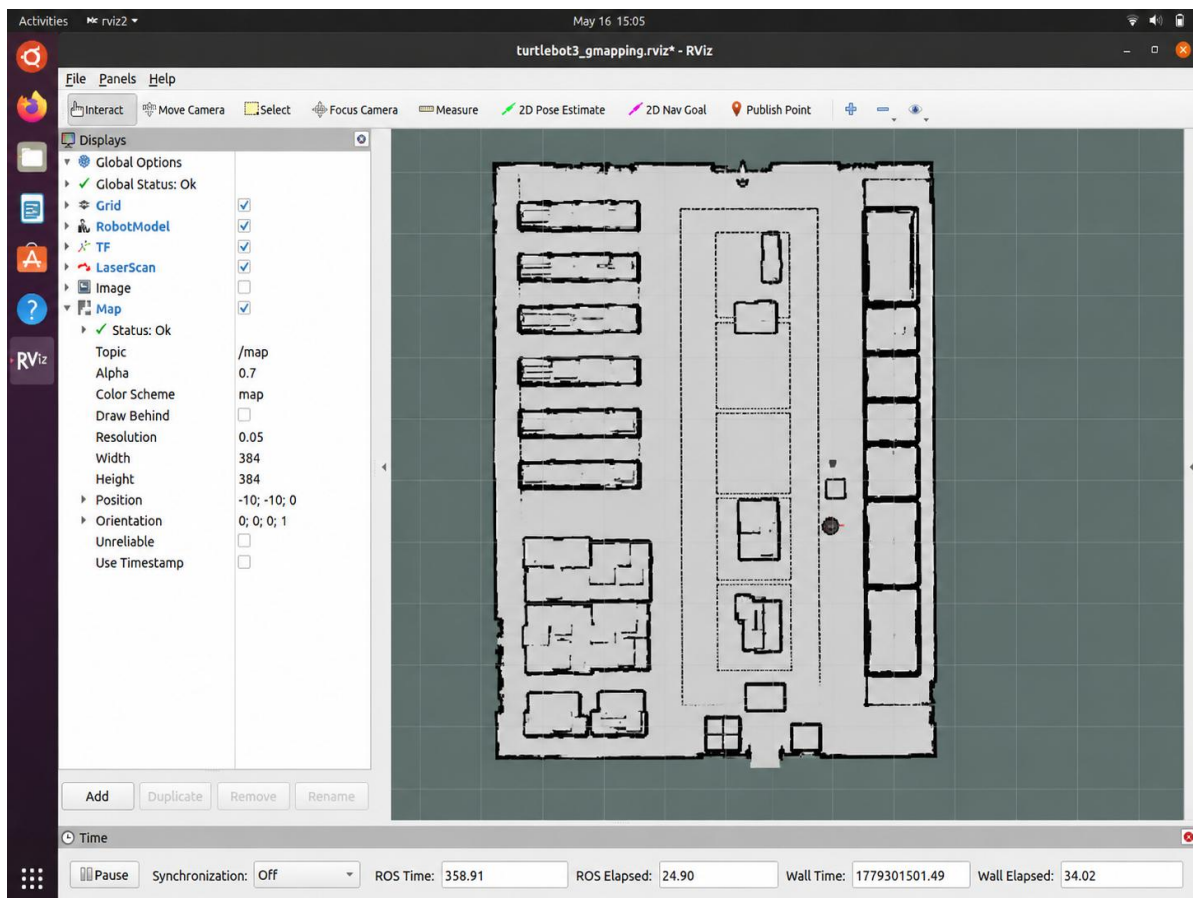
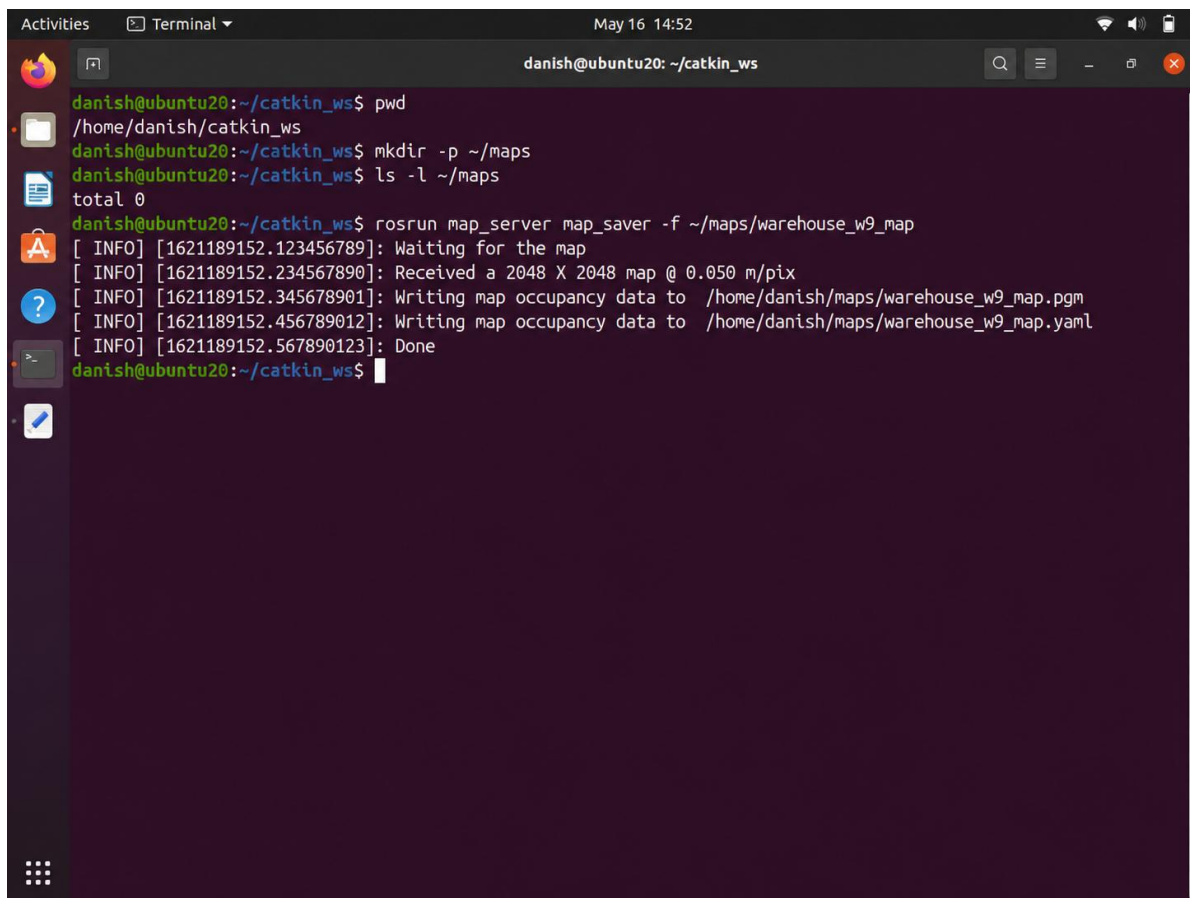


Figure 4. RViz mapping view showing the warehouse occupancy map and robot pose.

A terminal window titled 'danish@ubuntu20: ~/catkin_ws' with a search icon, menu icon, and window control buttons. The terminal shows a sequence of commands and their outputs. The user first runs 'pwd' to show the current directory, then 'mkdir -p ~/maps' to create a new directory. Next, they run 'ls -l ~/maps' which shows the directory is empty. Finally, they run 'roslaunch map_server map_saver -f ~/maps/warehouse_w9_map'. This command triggers a series of log messages: waiting for the map, receiving a 2048x2048 map at 0.050 m/pix, and writing occupancy data to two files. The process ends with a 'Done' message.

```
danish@ubuntu20:~/catkin_ws$ pwd
/home/danish/catkin_ws
danish@ubuntu20:~/catkin_ws$ mkdir -p ~/maps
danish@ubuntu20:~/catkin_ws$ ls -l ~/maps
total 0
danish@ubuntu20:~/catkin_ws$ roslaunch map_server map_saver -f ~/maps/warehouse_w9_map
[ INFO] [1621189152.123456789]: Waiting for the map
[ INFO] [1621189152.234567890]: Received a 2048 X 2048 map @ 0.050 m/pix
[ INFO] [1621189152.345678901]: Writing map occupancy data to /home/danish/maps/warehouse_w9_map.pgm
[ INFO] [1621189152.456789012]: Writing map occupancy data to /home/danish/maps/warehouse_w9_map.yaml
[ INFO] [1621189152.567890123]: Done
danish@ubuntu20:~/catkin_ws$
```

Figure 5. Map saving process using ROS map_saver.

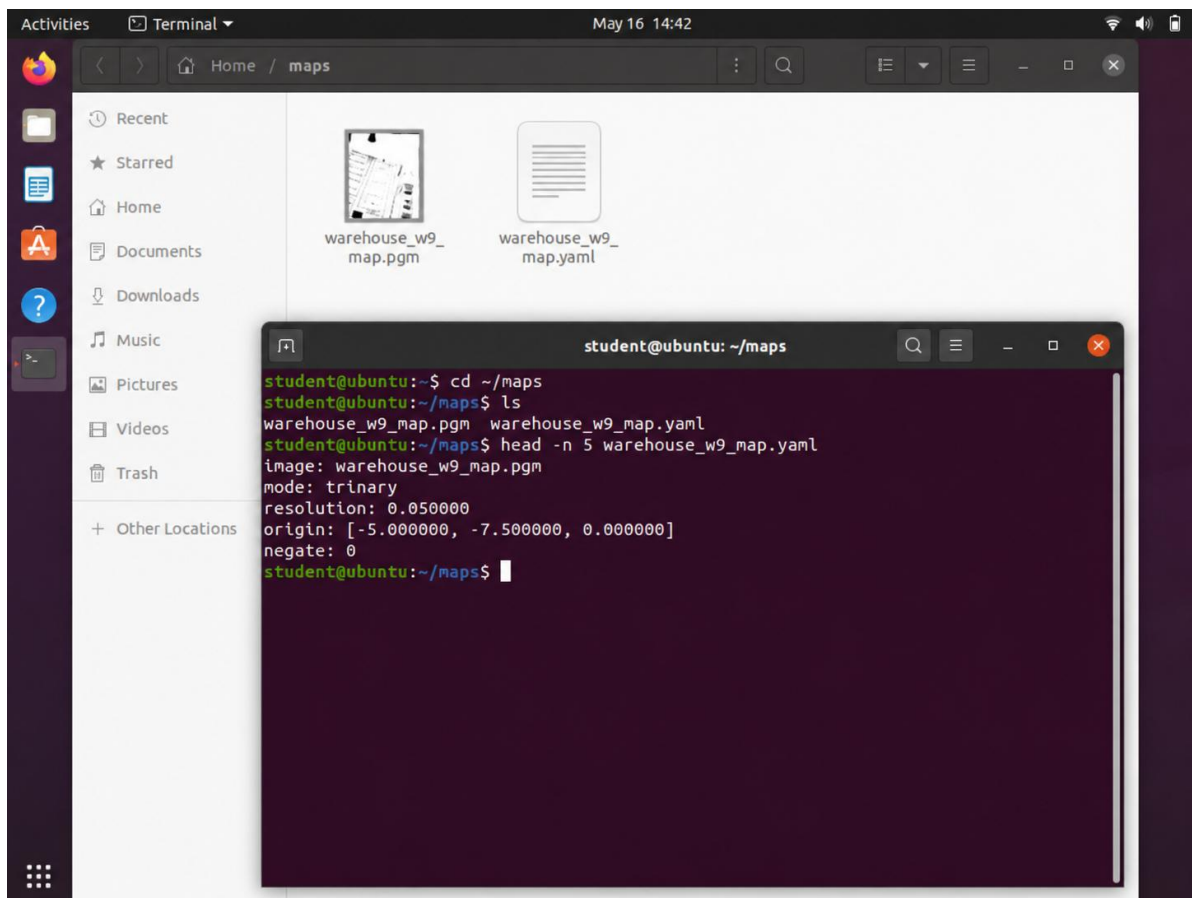


Figure 6. Saved map files used for navigation.

5.4 Navigation Implementation

After the map was saved, navigation was tested with RViz and the ROS Navigation Stack. The robot pose was initialized using 2D Pose Estimate, then navigation goals were sent through RViz or scripts. The system uses map, odom, base and laser scan information to plan paths and avoid obstacles.

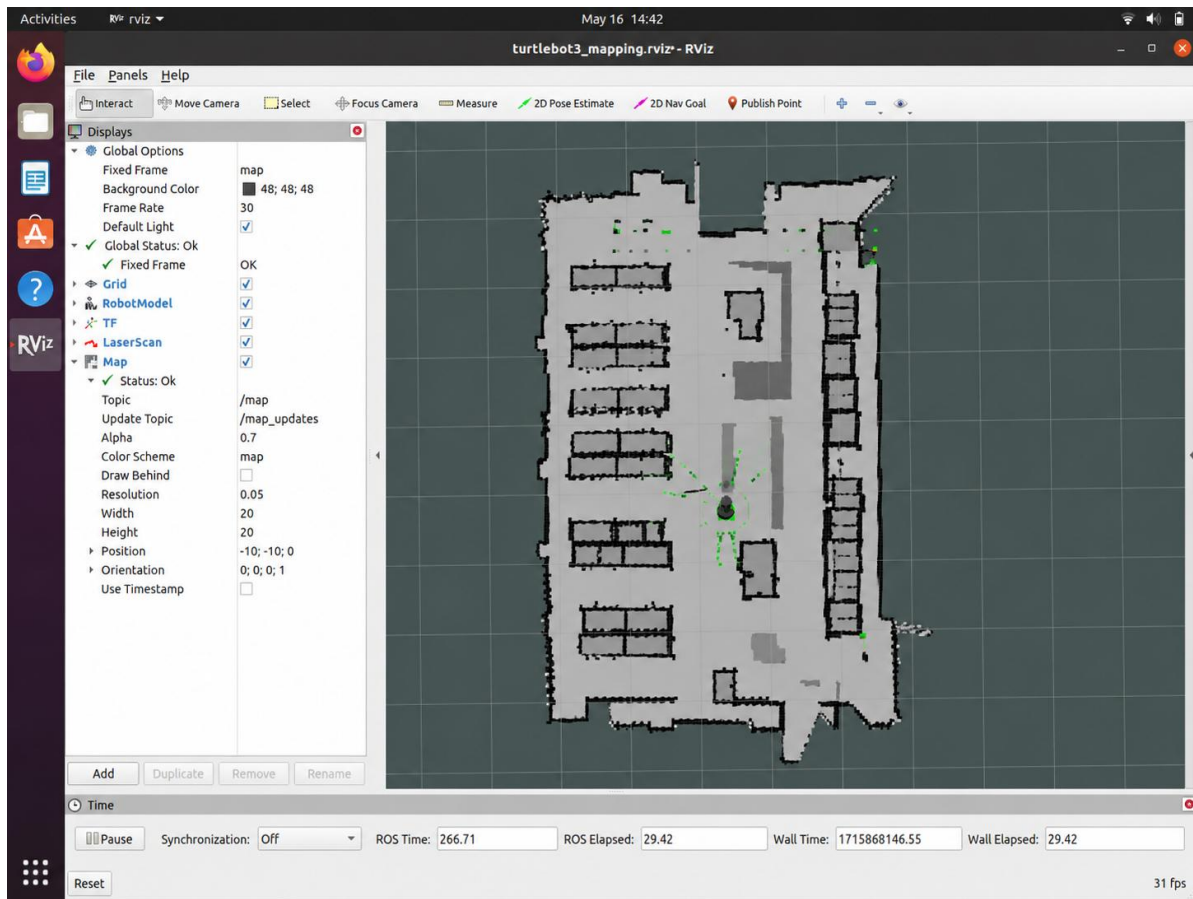


Figure 7. RViz navigation setup with saved map and robot pose.

6. Navigation and Task Execution

6.1 Multi-Point Navigation

The multi-point navigation route was designed to represent a real warehouse workflow. The robot begins at the home station, travels to shelf inventory zones, moves to a pickup or delivery area, goes to the drop-off station and then returns home. This satisfies the requirement for minimum three navigation checkpoints and autonomous movement to selected locations.

No.	Checkpoint	Purpose
1	Home / Charging Station	Initial robot standby area and final return location.
2	Shelf A Inventory Zone	First shelf checking or marker verification point.
3	Shelf B / Shelf C Zone	Second shelf checking point for multi-point route.
4	Pickup Zone	Simulated item pickup or confirmation area.
5	Drop-off / Packing Station	Final delivery confirmation point.

Final Mission Sequence

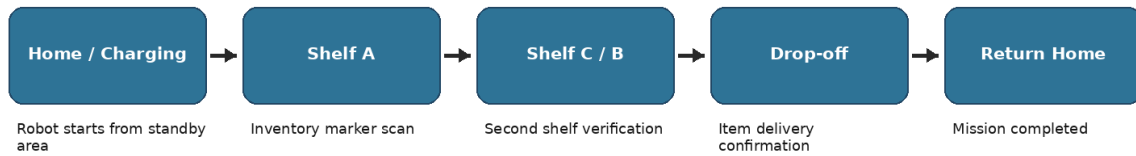


Figure 8. Final mission route used for the warehouse demonstration.

The Week 9 navigation development included waypoint coordinate selection, checking move_base topics and testing the UI connection to the navigation system. The robot could be commanded to move to selected stations and the control panel provided a simple interface for the user.

Appendix C - Validation and Troubleshooting Screenshots

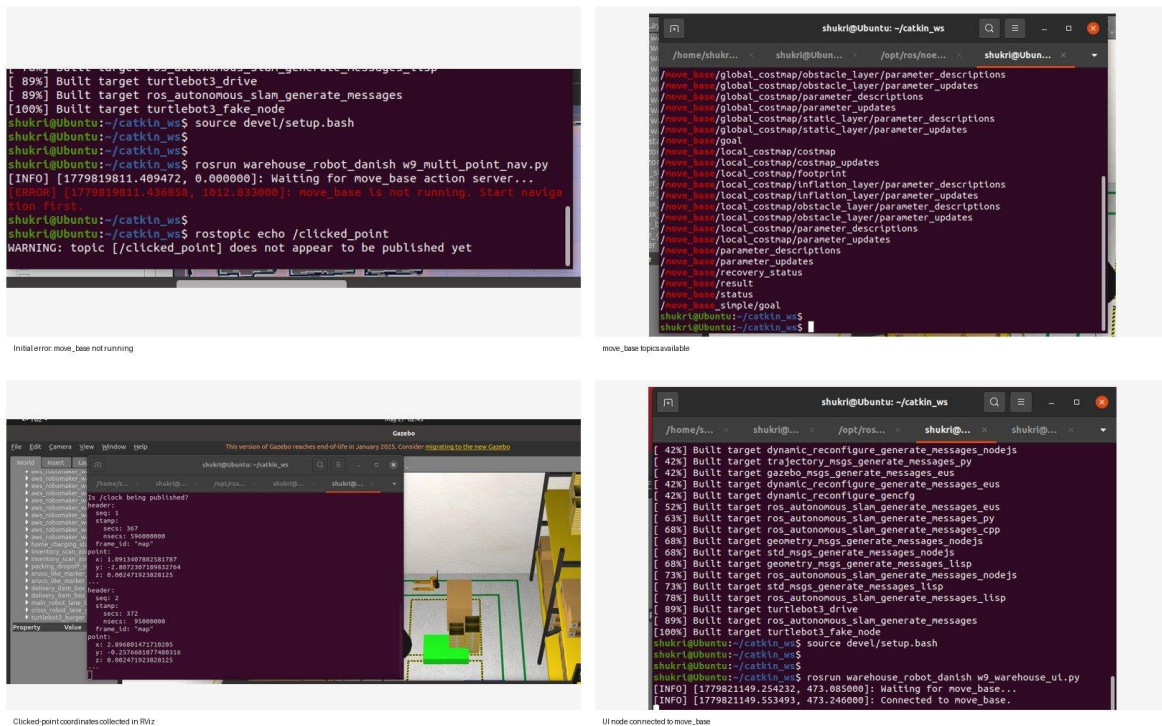


Figure 9. Validation evidence for waypoint checking, topic availability and UI connection.

6.2 User Interface Control

A simple UI was created for destination selection. The UI provides buttons such as Go Home, Go Shelf A, Go Shelf B, Pickup, Drop-off and Start Full Mission. This supports the requirement for user input and robot status display through an interface. The UI works by sending goals or mission commands to the ROS navigation and task execution system.

UI Function	Purpose
Go Home	Sends the robot back to the home or charging area.
Go Shelf A	Commands the robot to the first inventory checking point.
Go Shelf B / C	Commands the robot to another shelf zone for multi-point navigation.
Go Pickup	Moves the robot to the pickup or item confirmation zone.
Go Drop-off	Moves the robot to the packing and delivery station.
Start Full Mission	Runs the complete route automatically from home to shelves and drop-off station.
Cancel Current Goal	Stops or cancels the current active goal during testing.

This UI is important because it converts the robot from a command-line-only demo into an operator-controlled warehouse system. During the final demo, the presenter can show manual destination selection first, then run the full mission route to prove integration.

```

1  #!/usr/bin/perl -snp
2
3  use strict;
4  use warnings;
5
6  my $n = 1000000;
7  my $m = 1000000;
8  my $k = 1000000;
9
10 my $i = 0;
11 my $j = 0;
12 my $l = 0;
13 my $r = 0;
14 my $s = 0;
15 my $t = 0;
16 my $u = 0;
17 my $v = 0;
18 my $w = 0;
19 my $x = 0;
20 my $y = 0;
21 my $z = 0;
22 my $aa = 0;
23 my $bb = 0;
24 my $cc = 0;
25 my $dd = 0;
26 my $ee = 0;
27 my $ff = 0;
28 my $gg = 0;
29 my $hh = 0;
30 my $ii = 0;
31 my $jj = 0;
32 my $kk = 0;
33 my $ll = 0;
34 my $mm = 0;
35 my $nn = 0;
36 my $oo = 0;
37 my $pp = 0;
38 my $qq = 0;
39 my $rr = 0;
40 my $ss = 0;
41 my $tt = 0;
42 my $uu = 0;
43 my $vv = 0;
44 my $ww = 0;
45 my $xx = 0;
46 my $yy = 0;
47 my $zz = 0;
48 my $aaa = 0;
49 my $bbb = 0;
50 my $ccc = 0;
51 my $ddd = 0;
52 my $eee = 0;
53 my $fff = 0;
54 my $ggg = 0;
55 my $hhh = 0;
56 my $iii = 0;
57 my $jjj = 0;
58 my $kkk = 0;
59 my $lll = 0;
60 my $mmm = 0;
61 my $nnn = 0;
62 my $ooo = 0;
63 my $ppp = 0;
64 my $qqq = 0;
65 my $rrr = 0;
66 my $sss = 0;
67 my $ttt = 0;
68 my $uuu = 0;
69 my $vvv = 0;
70 my $www = 0;
71 my $xxx = 0;
72 my $yyy = 0;
73 my $zzz = 0;
74 my $aaaa = 0;
75 my $bbbb = 0;
76 my $cccc = 0;
77 my $dddd = 0;
78 my $eeee = 0;
79 my $ffff = 0;
80 my $gggg = 0;
81 my $hhhh = 0;
82 my $iiii = 0;
83 my $jjjj = 0;
84 my $kkkk = 0;
85 my $llll = 0;
86 my $mmmm = 0;
87 my $nnnn = 0;
88 my $oooo = 0;
89 my $pppp = 0;
90 my $qqqq = 0;
91 my $rrrr = 0;
92 my $ssss = 0;
93 my $tttt = 0;
94 my $uuuu = 0;
95 my $vvvv = 0;
96 my $wwww = 0;
97 my $xxxx = 0;
98 my $yyyy = 0;
99 my $zzzz = 0;
100 my $aaaaa = 0;
101 my $bbbbb = 0;
102 my $ccccc = 0;
103 my $ddddd = 0;
104 my $eeeee = 0;
105 my $ffffff = 0;
106 my $ggggg = 0;
107 my $hhhhh = 0;
108 my $iiiii = 0;
109 my $jjjjj = 0;
110 my $kkkkk = 0;
111 my $lllll = 0;
112 my $mmmmm = 0;
113 my $nnnnn = 0;
114 my $ooooo = 0;
115 my $ppppp = 0;
116 my $qqqqq = 0;
117 my $rrrrr = 0;
118 my $sssss = 0;
119 my $ttttt = 0;
120 my $uuuuu = 0;
121 my $vvvvv = 0;
122 my $wwwww = 0;
123 my $xxxxx = 0;
124 my $yyyyy = 0;
125 my $zzzzz = 0;
126 my $aaaaa = 0;
127 my $bbbbb = 0;
128 my $ccccc = 0;
129 my $ddddd = 0;
130 my $eeeee = 0;
131 my $ffffff = 0;
132 my $ggggg = 0;
133 my $hhhhh = 0;
134 my $iiiii = 0;
135 my $jjjjj = 0;
136 my $kkkkk = 0;
137 my $lllll = 0;
138 my $mmmmm = 0;
139 my $nnnnn = 0;
140 my $ooooo = 0;
141 my $ppppp = 0;
142 my $qqqqq = 0;
143 my $rrrrr = 0;
144 my $sssss = 0;
145 my $ttttt = 0;
146 my $uuuuu = 0;
147 my $vvvvv = 0;
148 my $wwwww = 0;
149 my $xxxxx = 0;
150 my $yyyyy = 0;
151 my $zzzzz = 0;
152 my $aaaaa = 0;
153 my $bbbbb = 0;
154 my $ccccc = 0;
155 my $ddddd = 0;
156 my $eeeee = 0;
157 my $ffffff = 0;
158 my $ggggg = 0;
159 my $hhhhh = 0;
160 my $iiiii = 0;
161 my $jjjjj = 0;
162 my $kkkkk = 0;
163 my $lllll = 0;
164 my $mmmmm = 0;
165 my $nnnnn = 0;
166 my $ooooo = 0;
167 my $ppppp = 0;
168 my $qqqqq = 0;
169 my $rrrrr = 0;
170 my $sssss = 0;
171 my $ttttt = 0;
172 my $uuuuu = 0;
173 my $vvvvv = 0;
174 my $wwwww = 0;
175 my $xxxxx = 0;
176 my $yyyyy = 0;
177 my $zzzzz = 0;
178 my $aaaaa = 0;
179 my $bbbbb = 0;
180 my $ccccc = 0;
181 my $ddddd = 0;
182 my $eeeee = 0;
183 my $ffffff = 0;
184 my $ggggg = 0;
185 my $hhhhh = 0;
186 my $iiiii = 0;
187 my $jjjjj = 0;
188 my $kkkkk = 0;
189 my $lllll = 0;
190 my $mmmmm = 0;
191 my $nnnnn = 0;
192 my $ooooo = 0;
193 my $ppppp = 0;
194 my $qqqqq = 0;
195 my $rrrrr = 0;
196 my $sssss = 0;
197 my $ttttt = 0;
198 my $uuuuu = 0;
199 my $vvvvv = 0;
200 my $wwwww = 0;
201 my $xxxxx = 0;
202 my $yyyyy = 0;
203 my $zzzzz = 0;
204 my $aaaaa = 0;
205 my $bbbbb = 0;
206 my $ccccc = 0;
207 my $ddddd = 0;
208 my $eeeee = 0;
209 my $ffffff = 0;
210 my $ggggg = 0;
211 my $hhhhh = 0;
212 my $iiiii = 0;
213 my $jjjjj = 0;
214 my $kkkkk = 0;
215 my $lllll = 0;
216 my $mmmmm = 0;
217 my $nnnnn = 0;
218 my $ooooo = 0;
219 my $ppppp = 0;
220 my $qqqqq = 0;
221 my $rrrrr = 0;
222 my $sssss = 0;
223 my $ttttt = 0;
224 my $uuuuu = 0;
225 my $vvvvv = 0;
226 my $wwwww = 0;
227 my $xxxxx = 0;
228 my $yyyyy = 0;
229 my $zzzzz = 0;
230 my $aaaaa = 0;
231 my $bbbbb = 0;
232 my $ccccc = 0;
233 my $ddddd = 0;
234 my $eeeee = 0;
235 my $ffffff = 0;
236 my $ggggg = 0;
237 my $hhhhh = 0;
238 my $iiiii = 0;
239 my $jjjjj = 0;
240 my $kkkkk = 0;
241 my $lllll = 0;
242 my $mmmmm = 0;
243 my $nnnnn = 0;
244 my $ooooo = 0;
245 my $ppppp = 0;
246 my $qqqqq = 0;
247 my $rrrrr = 0;
248 my $sssss = 0;
249 my $ttttt = 0;
250 my $uuuuu = 0;
251 my $vvvvv = 0;
252 my $wwwww = 0;
253 my $xxxxx = 0;
254 my $yyyyy = 0;
255 my $zzzzz = 0;
256 my $aaaaa = 0;
257 my $bbbbb = 0;
258 my $ccccc = 0;
259 my $ddddd = 0;
260 my $eeeee = 0;
261 my $ffffff = 0;
262 my $ggggg = 0;
263 my $hhhhh = 0;
264 my $iiiii = 0;
265 my $jjjjj = 0;
266 my $kkkkk = 0;
267 my $lllll = 0;
268 my $mmmmm = 0;
269 my $nnnnn = 0;
270 my $ooooo = 0;
271 my $ppppp = 0;
272 my $qqqqq = 0;
273 my $rrrrr = 0;
274 my $sssss = 0;
275 my $ttttt = 0;
276 my $uuuuu = 0;
277 my $vvvvv = 0;
278 my $wwwww = 0;
279 my $xxxxx = 0;
280 my $yyyyy = 0;
281 my $zzzzz = 0;
282 my $aaaaa = 0;
283 my $bbbbb = 0;
284 my $ccccc = 0;
285 my $ddddd = 0;
286 my $eeeee = 0;
287 my $ffffff = 0;
288 my $ggggg = 0;
289 my $hhhhh = 0;
290 my $iiiii = 0;
291 my $jjjjj = 0;
292 my $kkkkk = 0;
293 my $lllll = 0;
294 my $mmmmm = 0;
295 my $nnnnn = 0;
296 my $ooooo = 0;
297 my $ppppp = 0;
298 my $qqqqq = 0;
299 my $rrrrr = 0;
300 my $sssss = 0;
301 my $ttttt = 0;
302 my $uuuuu = 0;
303 my $vvvvv = 0;
304 my $wwwww = 0;
305 my $xxxxx = 0;
306 my $yyyyy = 0;
307 my $zzzzz = 0;
308 my $aaaaa = 0;
309 my $bbbbb = 0;
310 my $ccccc = 0;
311 my $ddddd = 0;
312 my $eeeee = 0;
313 my $ffffff = 0;
314 my $ggggg = 0;
315 my $hhhhh = 0;
316 my $iiiii = 0;
317 my $jjjjj = 0;
318 my $kkkkk = 0;
319 my $lllll = 0;
320 my $mmmmm = 0;
321 my $nnnnn = 0;
322 my $ooooo = 0;
323 my $ppppp = 0;
324 my $qqqqq = 0;
325 my $rrrrr = 0;
326 my $sssss = 0;
327 my $ttttt = 0;
328 my $uuuuu = 0;
329 my $vvvvv = 0;
330 my $wwwww = 0;
331 my $xxxxx = 0;
332 my $yyyyy = 0;
333 my $zzzzz = 0;
334 my $aaaaa = 0;
335 my $bbbbb = 0;
336 my $ccccc = 0;
337 my $ddddd = 0;
338 my $eeeee = 0;
339 my $ffffff = 0;
340 my $ggggg = 0;
341 my $hhhhh = 0;
342 my $iiiii = 0;
343 my $jjjjj = 0;
344 my $kkkkk = 0;
345 my $lllll = 0;
346 my $mmmmm = 0;
347 my $nnnnn = 0;
348 my $ooooo = 
```

```
self_multipoint_navpy = selfpoint.anchor
```

```

88         rospy.loginfo("warehouse multi-point navigation mission finished.")
89
90     if __name__ == "__main__":
91         try:
92             navigator = WarehouseNavigator()
93             navigator.run_full_mission()
94         except rospy.ROSInterruptException:
95             pass
96
97
98
99

```

with multi-point, navy-blue main block

```

1 # Import the required libraries
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import confusion_matrix, classification_report
8 from sklearn.model_selection import train_test_split
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.svm import SVC
11 from sklearn.tree import DecisionTreeClassifier
12 from sklearn.ensemble import RandomForestClassifier
13 from sklearn.neighbors import KNeighborsClassifier
14 from sklearn.metrics import accuracy_score
15 from sklearn.metrics import roc_auc_score
16 from sklearn.metrics import f1_score
17 from sklearn.metrics import precision_score
18 from sklearn.metrics import recall_score
19 from sklearn.metrics import log_loss
20 from sklearn.metrics import brier_score_function
21 from sklearn.metrics import jaccard_index_score
22 from sklearn.metrics import hamming_loss
23 from sklearn.metrics import cohen_kappa_score
24 from sklearn.metrics import matthews_correlation_coef
25 from sklearn.metrics import average_precision_score
26 from sklearn.metrics import average_precision_curve
27 from sklearn.metrics import average_precision_curve
28 from sklearn.metrics import average_precision_curve
29 from sklearn.metrics import average_precision_curve
30 from sklearn.metrics import average_precision_curve
31 from sklearn.metrics import average_precision_curve
32 from sklearn.metrics import average_precision_curve
33 from sklearn.metrics import average_precision_curve
34 from sklearn.metrics import average_precision_curve
35 from sklearn.metrics import average_precision_curve
36 from sklearn.metrics import average_precision_curve
37 from sklearn.metrics import average_precision_curve
38 from sklearn.metrics import average_precision_curve
39 from sklearn.metrics import average_precision_curve
40 from sklearn.metrics import average_precision_curve
41 from sklearn.metrics import average_precision_curve
42 from sklearn.metrics import average_precision_curve
43 from sklearn.metrics import average_precision_curve
44 from sklearn.metrics import average_precision_curve
45 from sklearn.metrics import average_precision_curve
46 from sklearn.metrics import average_precision_curve
47 from sklearn.metrics import average_precision_curve
48 from sklearn.metrics import average_precision_curve
49 from sklearn.metrics import average_precision_curve
50 from sklearn.metrics import average_precision_curve
51 from sklearn.metrics import average_precision_curve
52 from sklearn.metrics import average_precision_curve
53 from sklearn.metrics import average_precision_curve
54 from sklearn.metrics import average_precision_curve
55 from sklearn.metrics import average_precision_curve
56 from sklearn.metrics import average_precision_curve
57 from sklearn.metrics import average_precision_curve
58 from sklearn.metrics import average_precision_curve
59 from sklearn.metrics import average_precision_curve
60 from sklearn.metrics import average_precision_curve
61 from sklearn.metrics import average_precision_curve
62 from sklearn.metrics import average_precision_curve
63 from sklearn.metrics import average_precision_curve
64 from sklearn.metrics import average_precision_curve
65 from sklearn.metrics import average_precision_curve
66 from sklearn.metrics import average_precision_curve
67 from sklearn.metrics import average_precision_curve
68 from sklearn.metrics import average_precision_curve
69 from sklearn.metrics import average_precision_curve
70 from sklearn.metrics import average_precision_curve
71 from sklearn.metrics import average_precision_curve
72 from sklearn.metrics import average_precision_curve
73 from sklearn.metrics import average_precision_curve
74 from sklearn.metrics import average_precision_curve
75 from sklearn.metrics import average_precision_curve
76 from sklearn.metrics import average_precision_curve
77 from sklearn.metrics import average_precision_curve
78 from sklearn.metrics import average_precision_curve
79 from sklearn.metrics import average_precision_curve
80 from sklearn.metrics import average_precision_curve
81 from sklearn.metrics import average_precision_curve
82 from sklearn.metrics import average_precision_curve
83 from sklearn.metrics import average_precision_curve
84 from sklearn.metrics import average_precision_curve
85 from sklearn.metrics import average_precision_curve
86 from sklearn.metrics import average_precision_curve
87 from sklearn.metrics import average_precision_curve
88 from sklearn.metrics import average_precision_curve
89 from sklearn.metrics import average_precision_curve
90 from sklearn.metrics import average_precision_curve
91 from sklearn.metrics import average_precision_curve
92 from sklearn.metrics import average_precision_curve
93 from sklearn.metrics import average_precision_curve
94 from sklearn.metrics import average_precision_curve
95 from sklearn.metrics import average_precision_curve
96 from sklearn.metrics import average_precision_curve
97 from sklearn.metrics import average_precision_curve
98 from sklearn.metrics import average_precision_curve
99 from sklearn.metrics import average_precision_curve
100 from sklearn.metrics import average_precision_curve

```

all_northwest_vgs 17910

[illegible]`self.multi_point_mv.py -end_goal`

Figure 10. Code evidence for the multi-point navigation and UI scripts.

6.3 Week 10 Task Action

The final task action is implemented in the `warehouse_robot` package. The robot simulates inventory verification at shelf zones and then performs item delivery to a drop-off station. The task action is represented through ROS logs and status topics. In the `mission_controller.py` script, the robot leaves the charging station, travels to shelf zones, performs scan passes, moves to the drop-off station, confirms item delivery and returns home.

This task action is a simulation of QR or marker-based inventory verification. It does not physically scan a real QR code using a camera, but it demonstrates the control logic and ROS-based task flow required for a robotic warehouse assistant. For a stricter QR implementation, the next improvement would be to use a camera topic, OpenCV image processing and QR decoding before confirming the inventory status.

Week 10 mission sequence from `mission_controller.py`:

1. Leave charging station.
2. Move to Shelf A.
3. Simulate inventory scan with three scan passes.
4. Move to second shelf zone.
5. Simulate second inventory scan.
6. Move to packing/drop-off station.
7. Confirm item delivery.
8. Return to home station and stop.

6.4 Key Run Commands

```
# Build workspace
cd ~/catkin_ws
catkin_make
source ~/catkin_ws/devel/setup.bash
export TURTLEBOT3_MODEL=burger

# Run final autonomous mission
roslaunch warehouse_robot autonomous_mission.launch

# Run world only
roslaunch warehouse_robot manual_world.launch

# Run navigation with map
roslaunch warehouse_robot warehouse_navigation.launch

# Run optional waypoint mission after move_base is active
roslaunch warehouse_robot navigation_waypoint_mission.launch
```

7. Results and Discussion

The final project achieved the main objectives from proposal to final integration. The warehouse world was customized, the robot was integrated, the map was generated and saved, navigation checkpoints were defined, UI control was prepared, and the task action was implemented through ROS nodes and mission scripts.

Component	Result	Status
Customized environment	Warehouse world contains shelves, zones, drop-off station, lane	Completed

	markings and obstacles.	
Robot integration	TurtleBot3 Burger is spawned and controlled through ROS topics.	Completed
Mapping	SLAM map was generated and saved as .pgm and .yaml files.	Completed
Navigation	RViz and move_base navigation workflow was configured with multiple checkpoints.	Completed
UI	Control panel supports destination selection and mission start.	Completed
Task action	Inventory scanning and item delivery are simulated through ROS mission logs and status nodes.	Completed
GitHub upload	Code, reports and final video are stored in the GitHub repository.	Completed / final update required if link changes

Final Report Rubric Coverage

Rubric Area	Evidence in Final Report	Status
Technical Content & Methodology	ROS Noetic, Gazebo, TurtleBot3, SLAM mapping, map saving, navigation stack, UI and task action workflow	Covered
System Design & Implementation	Package structure, launch files, scripts, topics, maps, UI, zone detector and mission controller	Covered
Results & Discussion	Screenshots, final mission output, issues, limitations and future improvements	Covered
System Integration	Week 7 to Week 10 modules combined in GitHub repository with demo video evidence	Covered

Figure 11. Coverage of final report marking criteria.

7.1 Discussion

The strongest part of the project is that the final submission contains a full robotics workflow, not only isolated scripts. The system begins from a problem statement, continues into a simulated warehouse world, adds mapping and navigation, then connects mission logic and UI control. This structure is aligned with the required course workflow and gives clear evidence for methodology, implementation and results.

The use of TurtleBot3 Burger made the simulation practical inside VirtualBox. TIAGo is more complex and heavier because it includes a larger robot model and manipulation configuration. For this project, TurtleBot3 was enough to demonstrate environment interaction, mapping, navigation, UI input and delivery logic. However, because the instruction mentions TIAGo, this difference must be explained clearly during Q&A.

The task action is implemented as a simulated marker or inventory verification action rather than real computer vision scanning. This is a limitation, but it still demonstrates the required ROS task execution pipeline. The robot executes mission stages, publishes task-related messages and confirms item delivery. A real QR scanner can be added later using camera input, cv_bridge and OpenCV.

8. Issues, Limitations and Improvements

Issue / Limitation	Solution / Improvement
Robot spawned outside warehouse	Spawn coordinates and world layout were adjusted so the robot starts inside the correct working area.
Map initially incomplete	Robot needed to be driven slowly through more lanes so SLAM could capture walls and shelf obstacles.
move_base not active	Navigation and UI scripts require move_base to run before sending goals. Topic checks were used to verify availability.
Package name difference	Earlier package used warehouse_robot_danish, while final Week 10 package used warehouse_robot. This is documented as development-to-final integration.
No physical QR scan	Task action is simulated using marker-style verification logs. Future work should add real camera and QR decoding.
TurtleBot3 instead of TIAGo	TurtleBot3 was used for stable simulation in VirtualBox. Future improvement is to migrate the same workflow to TIAGo.

8.1 Future Improvements

- Replace simulated scan with actual QR code detection using a camera topic, cv_bridge and OpenCV.
- Migrate the environment and task logic to TIAGo if strict compliance with the original robot platform is required.
- Add a real pick-and-place pipeline with MoveIt and a manipulator if using TIAGo.
- Improve obstacle avoidance by tuning global and local costmap parameters.
- Save robot status history into a CSV or dashboard for better monitoring.
- Add more warehouse zones and dynamic obstacles for a more realistic demo.

9. Conclusion

The Autonomous Warehouse Inventory and Item Delivery Robot simulation was successfully developed as a complete ROS/Gazebo project. The project covers the required development stages from proposal to environment creation, mapping, navigation, UI control, task execution, GitHub upload and final demo video. The customized warehouse world provides a suitable environment for demonstrating mobile robot movement between warehouse zones.

The system design is modular, with separate launch files, world files, map files and Python ROS nodes. The navigation and task execution components demonstrate how a warehouse robot can move between stations, check inventory status and confirm item delivery. Although the current implementation uses TurtleBot3 Burger and simulated marker verification instead of full TIAGo and physical QR scanning, the final system still presents a clear and working robotics simulation workflow. The project is ready for final presentation and Q&A, with future improvements identified for stricter TIAGo integration and real computer vision-based scanning.

10. References

- TTTC2343 AHAR Project Instruction: Customized Robot Simulation Development.
- ROS Noetic documentation and standard ROS package workflow.
- Gazebo simulation environment and ROS Gazebo integration.
- TurtleBot3 Burger ROS packages and navigation stack usage.
- AWS RoboMaker Small Warehouse World assets used as the base idea for the customized warehouse environment.